

CESED – CENTRO DE ENSINO SUPERIOR E DESENVOLVIMENTO

CENTRO UNIVERSITÁRIO UNIFACISA

CURSO - SISTEMAS DE INFORMAÇÃO

PROFESSORA MARCELLA MEDEIROS

COMPETÊNCIA

CONNECTAR BANCO DE DADOS COM POO

**PROJETO INTERDISCIPLINAR: SISTEMA DE GESTÃO PARA CLÍNICA DE
ESPECIALIDADES**

Samuel Gomes Soares

João Victor Silva Guedes

Naldo Junior Santos de Lima

Gabriel Barbosa

Campina Grande/PB

Dezembro 2025

1. INTRODUÇÃO E CONTEXTUALIZAÇÃO

O gerenciamento manual ou descentralizado de clínicas médicas frequentemente resulta em ineficiências operacionais, como conflitos de agenda (duplicidade de horários), dificuldade no controle de convênios atendidos por cada profissional e ausência de registros históricos de alterações. Esses problemas impactam diretamente a qualidade do atendimento ao paciente e a organização financeira da instituição.

O presente projeto propõe o desenvolvimento de um sistema informatizado para a gestão de agendamentos de uma Clínica de Especialidades Médicas. O objetivo principal é estruturar uma base de dados robusta, normalizada e íntegra, capaz de sustentar as operações diárias da clínica, garantindo a consistência das informações e a segurança no acesso aos dados.

O escopo desta primeira fase abrange o levantamento de requisitos, a definição das regras de negócio e a modelagem do banco de dados (Conceitual e Lógico), utilizando o SGBD MySQL.

2. ESPECIFICAÇÃO DE REQUISITOS

Para o desenvolvimento do sistema, foram mapeadas as necessidades dos três principais perfis de usuários: Administradores (gestão), Médicos (atendimento) e Pacientes (agendamento).

2.1. Requisitos Funcionais (RF)

Os requisitos funcionais descrevem as interações e funcionalidades esperadas do sistema:

- **[RF001] Controle de Acesso e Autenticação:** O sistema deve permitir o cadastro de usuários e validar o acesso mediante login e senha criptografada, diferenciando perfis de acesso.

- **[RF002] Manter Médicos:** Deve ser possível cadastrar, editar e consultar médicos, registrando dados profissionais (CRM) e pessoais.
- **[RF003] Manter Pacientes:** O sistema deve gerenciar o cadastro de pacientes, incluindo dados demográficos e vínculo com convênio titular.
- **[RF004] Gestão de Especialidades e Convênios:** Permitir o cadastro das especialidades oferecidas e dos planos de saúde aceitos pela clínica.
- **[RF005] Associação de Profissionais:** O sistema deve permitir vincular quais especialidades um médico possui e quais convênios ele atende, resolvendo a relação de muitos-para-muitos.
- **[RF006] Agendamento de Consultas:** Funcionalidade central que permite marcar consultas vinculando paciente, médico, especialidade e data/hora.
- **[RF007] Cancelamento e Status:** Permitir a alteração do status da consulta (Agendada, Confirmada, Realizada, Cancelada).
- **[RF008] Auditoria de Dados:** O sistema deve manter, obrigatoriamente, um registro histórico (log) de todas as modificações de status das consultas.

2.2. Requisitos Não Funcionais (RNF)

Os requisitos não funcionais definem os critérios de qualidade e restrições técnicas:

- **[RNF001] Integridade Referencial:** O banco de dados deve utilizar chaves estrangeiras (Foreign Keys) para garantir a consistência dos relacionamentos.
- **[RNF002] Normalização:** O modelo de dados deve respeitar, no mínimo, a 3ª Forma Normal (3FN) para evitar redundâncias.
- **[RNF003] Segurança de Dados:** As senhas devem ser armazenadas utilizando hash e nunca em texto plano.
- **[RNF004] SGBD:** O projeto será implementado utilizando MySQL/MariaDB.

3. REGRAS DE NEGÓCIO (RN)

As regras de negócio definem as restrições lógicas que o sistema deve impor para refletir a realidade operacional da clínica:

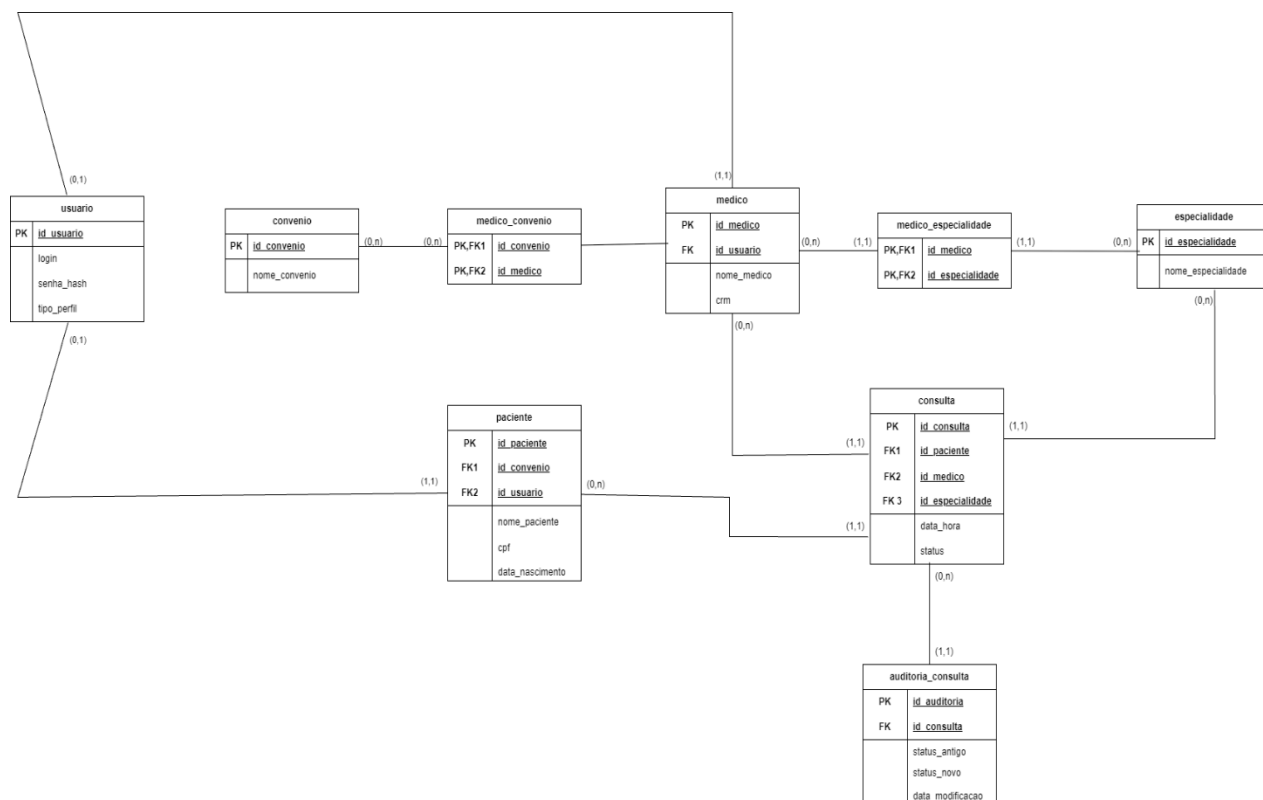
- **[RN01] Unicidade de Identificação:** Não será permitido o cadastro de dois médicos com o mesmo CRM ou dois pacientes com o mesmo CPF.
- **[RN02] Disponibilidade Médica:** Um médico não pode ter duas consultas agendadas para o mesmo horário (conflito de agenda).
- **[RN03] Disponibilidade do Paciente:** Um paciente não pode estar agendado em duas consultas simultâneas.
- **[RN04] Validação de Convênio:** O agendamento só poderá ser efetivado se o médico selecionado atender o convênio do paciente.
- **[RN05] Pertinência da Especialidade:** Um médico só pode realizar consultas nas especialidades às quais está formalmente vinculado no sistema.
- **[RN06] Imutabilidade do Histórico:** Registros de auditoria não podem ser alterados ou excluídos pelos usuários do sistema.

4. PROJETO DE BANCO DE DADOS

A modelagem dos dados foi realizada visando a escalabilidade e a integridade das informações. Abaixo, detalha-se a estrutura adotada.

4.1 Modelo Entidade-Relacionamento (DER)

O diagrama abaixo ilustra as entidades, seus atributos e os relacionamentos estabelecidos. Destaca-se a resolução dos relacionamentos N:N através de entidades associativas e a inclusão da tabela de auditoria e controle de usuários.



4.1 Dicionário de Dados e Decisões de Modelagem

Para atender aos requisitos levantados, a estrutura física do banco de dados foi organizada da seguinte forma:

- **Segurança (usuario):** Tabela criada para centralizar as credenciais de acesso. As tabelas **medico** e **paciente** possuem chaves estrangeiras apontando para **usuario**, permitindo que os perfis acessem o sistema.
- **Entidades Fortes (medico, paciente, convenio, especialidade):** Armazenam os dados cadastrais principais. Campos únicos (UNIQUE) foram aplicados em CRM, CPF e Nomes de Convênios para garantir a [RN01].
- **Tabelas Associativas (medico_especialidade, medico_convenio):** Criadas para resolver o relacionamento "Muitos-para-Muitos". Isso permite que um médico tenha múltiplas especialidades e atenda múltiplos convênios, e vice-versa, garantindo a normalização do banco.

- **Processo Principal (consulta):** Tabela central que conecta todas as anteriores. Possui restrições UNIQUE compostas (id_medico, data_hora) e (id_paciente, data_hora) para impedir agendamentos duplicados, satisfazendo as regras [RN02] e [RN03].
- **Auditoria (auditoria_consulta):** Tabela destinada ao log de transações. Ela não possui integridade referencial restritiva (ON DELETE CASCADE) propositalmente, para assegurar que o histórico de alterações permaneça acessível mesmo que os registros originais sejam removidos, atendendo à [RN06].

CONSIDERAÇÕES FINAIS DA FASE 1

A etapa de modelagem e estruturação do banco de dados foi concluída com sucesso. O esquema desenvolvido no MySQL atende a todos os requisitos funcionais e regras de negócio propostos, estabelecendo uma base sólida para a implementação das lógicas de automação (Triggers, Procedures e Funções) e desenvolvimento da aplicação backend nas próximas fases do projeto.

5. IMPLEMENTAÇÃO DA LÓGICA DE NEGÓCIO NO BANCO DE DADOS (FASE 2)

Após a consolidação da estrutura de dados na primeira fase, o projeto avançou para a implementação da "inteligência" do sistema diretamente no SGBD (Sistema Gerenciador de Banco de Dados). O objetivo desta etapa foi migrar regras de negócio críticas, que anteriormente poderiam residir apenas na aplicação, para dentro do banco de dados MySQL.

Essa abordagem garante que as regras de integridade e validação sejam respeitadas independentemente da interface utilizada (seja via aplicação Web, Mobile ou acesso direto via terminal), aumentando significativamente a segurança e a consistência dos dados.

Abaixo, detalham-se as rotinas desenvolvidas:

5.1. Funções Armazenadas (Stored Functions)

As funções foram criadas para encapsular cálculos e verificações repetitivas, retornando valores que auxiliam na tomada de decisão dos procedimentos maiores.

- **Função:** fn_verificar_disponibilidade(p_id_medico, p_data_hora)
- **Objetivo:** Verificar se um médico possui horário livre em determinada data e hora.
- **Regra de Negócio Atendida:** [RN02] Disponibilidade Médica.
- **Lógica Implementada:** A função realiza uma contagem na tabela consulta. Se encontrar qualquer registro para o médico e horário informados que não esteja com status 'CANCELADA', retorna FALSE (0), indicando ocupação. Caso contrário, retorna TRUE (1).
- **Uso no Sistema:** É invocada internamente pela Stored Procedure de agendamento antes de qualquer tentativa de inserção.
- **Função:** fn_faturamento_medico(p_id_medico)
- **Objetivo:** Calcular o montante financeiro gerado por um profissional.
- **Lógica Implementada:** Realiza a soma do campo valor de todas as consultas vinculadas ao médico cujo status seja 'REALIZADA'. Trata valores nulos retornando zero, garantindo consistência em relatórios financeiros.

5.2. Gatilhos (Triggers)

Os gatilhos foram configurados para disparar ações automáticas de validação e auditoria, garantindo que o sistema reaja a eventos de inserção e atualização sem intervenção manual.

- **Trigger:** trg_validar_convenio_medico
- **Tipo:** BEFORE INSERT (Antes da Inserção) na tabela consulta.

- **Objetivo:** Impedir agendamentos inválidos quanto à cobertura de planos de saúde.
- **Regra de Negócio Atendida:** [RN04] Validação de Convênio.
- **Fluxo de Execução:** Ao receber uma solicitação de nova consulta, o gatilho intercepta a operação e verifica na tabela associativa `medico_convênio` se existe vínculo entre o médico selecionado e o convênio do paciente. Se o vínculo não existir, o gatilho aborta a transação e retorna uma mensagem de erro personalizada ao usuário, impedindo a gravação de dados inconsistentes.
- **Trigger:** `trg_log_auditoria`
- **Tipo:** AFTER UPDATE (Depois da Atualização) na tabela consulta.
- **Objetivo:** Garantir a rastreabilidade das informações.
- **Regra de Negócio Atendida:** [RN06] Imutabilidade do Histórico e [RF008] Auditoria.
- **Fluxo de Execução:** O gatilho monitora alterações na tabela. Sempre que o status de uma consulta é modificado (ex.: de 'AGENDADA' para 'CANCELADA'), o sistema captura automaticamente o estado anterior (OLD), o novo estado (NEW), a data da ocorrência e o usuário responsável, gravando esses dados na tabela `auditoria_consulta`.

4.1 Procedimentos Armazenados (Stored Procedures)

Os procedimentos foram desenvolvidos para executar transações de negócio completas, orquestrando as funções e garantindo que operações complexas sejam tratadas de forma atômica.

- **Procedure:** `prc_agendar_consulta`
- **Objetivo:** Realizar o agendamento de consultas de forma segura e validada.
- **Fluxo de Execução:**
 1. Recebe os parâmetros da consulta (paciente, médico, especialidade, data, valor).

2. Executa a função `fn_verificar_disponibilidade`. Se o retorno for negativo, lança uma exceção de "Horário Indisponível" e encerra o processo.
3. Se disponível, prossegue para o comando `INSERT`. Neste momento, a trigger `trg_validar_convenio_medico` é acionada automaticamente para validar o plano de saúde.
4. Se todas as validações passarem, o registro é efetivado e uma mensagem de sucesso é retornada.

6. DESENVOLVIMENTO DO BACKEND (API)

Para consumir a estrutura de banco de dados criada, foi desenvolvida uma aplicação Backend utilizando a tecnologia Node.js com o framework Express. A escolha dessa stack deve-se à sua natureza assíncrona e à eficiência na construção de APIs RESTful.

6.1. Arquitetura da Aplicação

A aplicação foi estruturada seguindo o padrão MVC simplificado, com separação clara de responsabilidades:

- **Configuração de Banco (`db.js`):** Módulo responsável exclusivamente pela conexão com o MySQL, utilizando Pool de Conexões para garantir performance em múltiplos acessos simultâneos.
- **Camada de Rotas e Controladores (`index.js`):** Define os endpoints (URLs) disponíveis e a lógica de recepção e resposta das requisições HTTP.

6.2. Integração com o Banco de Dados

Diferentemente de aplicações tradicionais que trazem toda a regra de negócio para o código (Backend), este projeto adotou a estratégia de delegar as regras

de integridade ao Banco de Dados. O Backend atua como uma interface de comunicação.

Principais Endpoints implementados

:

1. POST /agendar:

- Este endpoint não realiza verificações de horário ou convênio no código JavaScript. Ele recebe o JSON com os dados e invoca diretamente a procedure CALL prc_agendar_consulta(...).
- O tratamento de erros captura as exceções lançadas pelo MySQL (pelas triggers ou procedures) e as repassa ao Frontend com o código HTTP adequado (400 Bad Request).

2. GET /medicos/:id/faturamento:

- Endpoint analítico que executa a função fn_faturamento_medico via instrução SQL SELECT, demonstrando a capacidade de integração de rotinas de cálculo.

3. CRUD de Pacientes (/pacientes):

- Implementação completa das operações de Criação, Leitura, Atualização e Exclusão para gestão cadastral, demonstrando a manipulação direta de tabelas quando não há regras de negócio complexas envolvidas.

7. CONSIDERAÇÕES FINAIS

O desenvolvimento deste projeto permitiu a aplicação prática dos conceitos de banco de dados relacional e desenvolvimento web. A divisão em duas fases evidenciou a importância de uma modelagem sólida (Fase 1) para suportar regras de negócio complexas (Fase 2).

A centralização das regras no banco de dados, através de Triggers e Procedures, provou ser uma estratégia eficiente para garantir a integridade dos dados, evitando que erros de aplicação ou acessos indevidos corrompam o estado do sistema. A integração com o Backend Node.js demonstrou como aplicações modernas podem interagir de forma performática com bancos SQL legados ou robustos, respeitando as camadas de responsabilidade de cada tecnologia.

O resultado final é um sistema funcional, auditável e seguro, apto a solucionar os problemas de gestão de agendamentos propostos no início do trabalho.

REFERÊNCIAS BIBLIOGRÁFICAS

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7. ed. São Paulo: Pearson, 2018. DATE, C. J. **Introdução a Sistemas de Bancos de Dados**. 8. ed. Rio de Janeiro: Elsevier, 2004. DOCUMENTAÇÃO MYSQL. **MySQL 8.0 Reference Manual**. Disponível em: <https://dev.mysql.com/doc/>.